

# 3.8 Remembering Things

---

The chapter's capstone

## WHAT THIS LECTURE DOES

Takes the raw file I/O from 3.7 and wraps it in functions, then uses them properly:

1. **Load** whatever friends are already saved
2. **Show** them
3. Let the user **add more**
4. **Save** the combined list back

The friends list grows a little more every time you run the program.

## WRAPPING FILE I/O IN A FUNCTION



```
std::vector<std::string> loadFriendsFromFile(const std::string& fileName) {  
    std::vector<std::string> friends;  
    std::ifstream inputFile(fileName);  
  
    std::string friendName;  
    while (std::getline(inputFile, friendName)) {  
        friends.push_back(friendName);  
    }  
  
    return friends;  
}
```

## THE OTHER HALF



```
void saveFriendsToFile(const std::vector<std::string>& friends,
                      const std::string& fileName) {
    std::ofstream outputFile(fileName);
    for (const std::string& friendName : friends) {
        outputFile << friendName << "\n";
    }
    std::println("Saved {} friend(s) to {}", friends.size(), fileName);
}
```

## LOAD FIRST, THEN COLLECT MORE



```
std::vector<std::string> friends = loadFriendsFromFile(fileName);

std::println("\nYou already have {} friend(s) saved:", friends.size());
for (const std::string& friendName : friends) {
    std::println(" - {}", friendName);
}

std::vector<std::string> newFriends = collectMoreFriendNames();
friends.insert(friends.end(), newFriends.begin(), newFriends.end());

saveFriendsToFile(friends, fileName);
```

### 3.8 / RUN IT TWICE

## PROOF IT PERSISTS

### First run

`friends.txt` doesn't exist yet



```
You already have 0  
friend(s) saved:
```

### Second run

picks up right where you left off



```
You already have 2  
friend(s) saved:  
- Grace  
- Hopper
```

WHAT THIS PROGRAM ACTUALLY NEEDED

| NEED                        | C++ BUILDING BLOCK                  |
|-----------------------------|-------------------------------------|
| Say something to the screen | Output                              |
| Do a named, reusable action | A function                          |
| Ask the user something      | Input                               |
| Compute something new       | Arithmetic                          |
| Follow a different path     | <code>if</code> / <code>else</code> |
| Repeat until told to stop   | A loop                              |
| Hold a growing list         | <code>std::vector</code>            |
| Remember across runs        | File I/O                            |

## WHAT C++'S LATEST FEATURES BOUGHT US

- `std::print` / `std::println` - a single `{}`-style format string instead of chained `std::cout <<` - readable from lecture one
- **Brace initialization** (`{}`) - `int age{};` refuses to silently truncate a value you didn't mean to lose - the same habit chapter 4 explains the **why** of

None of this was necessary to teach the underlying ideas - but it made the code you typed along the way noticeably less ceremonial.



# End of Chapter 3

---

First Steps

**Next: Chapter 4 — Variables and Data Types**